🛡️ Proposta de Projeto: Detector de Intrusão (IDS) para a FATEC Mogi das Cruzes

Disciplina: Segurança da Informação, 5º semestre de ADS

Grupo: Andrey Marques · Enzo Kuwamoto · Henrique Hida · Matheus Schalch · Thomas Moises

Entrega: 06/11

---

1. A Ideia (Rascunho)

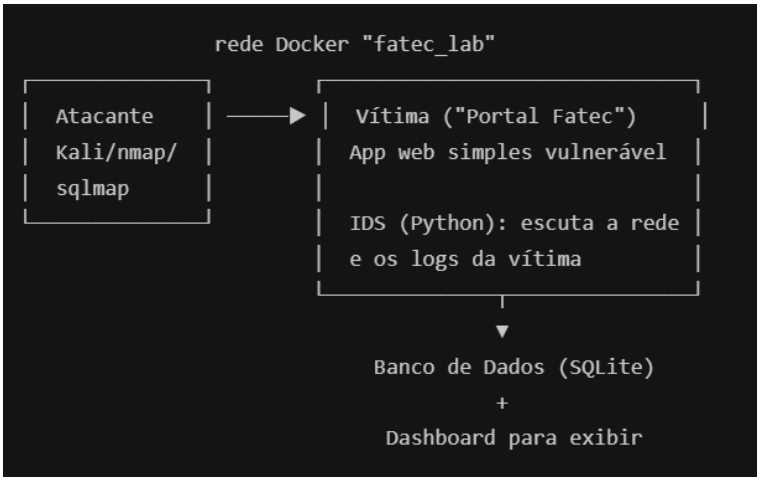
Pelo que vi, o professor curte quando a gente coloca a mão no código. Então a sugestão é: construirmos em **Python** um **IDS (Detector de Intrusão) próprio**, rodando num laboratório simulado em **Docker**. Ele monitoraria 4 tipos de ataques contra um "Portal Fatec" falso e mostraria os alertas numa telinha (dashboard).

Por que esse caminho?

- Mostra que sabemos programar (não apenas instalar um Snort/Suricata pronto).
- Com o Docker, o ambiente roda igual no PC de todo mundo com um comando só.
- Dá para dividir 1 tipo de ataque para cada pessoa. Assim ninguém fica travado dependendo do outro.

---

2. Como funcionaria (Arquitetura)



A base teórica por trás disso envolve o formato de regras de ferramentas de mercado (Snort/Suricata) e a base de ataques **MITRE ATT&CK** (para darmos aquela valorizada no trabalho). Basicamente, o IDS vai contar eventos suspeitos por IP em uma janela de tempo e gerar um alerta.

Infraestrutura Docker (Onde o código roda)

O projeto será orquestrado via `docker-compose`, subindo três contêineres distintos. Para que o IDS consiga monitorar a Vítima de forma transparente, utilizaremos os seguintes conceitos do Docker:

- **Network Namespace Compartilhado (`network_mode: "service:vitima"`):** Contêineres não enxergam o tráfego de rede uns dos outros. Para resolver isso, o contêiner do `ids` será configurado para compartilhar a mesma interface de rede do contêiner `vitima`. Assim, todo tráfego TCP/IP destinado à Vítima passará por dentro do IDS, permitindo que a biblioteca **Scapy** intercepte os pacotes.
- **Privilégios de Rede (`cap_add: [NET_RAW, NET_ADMIN]`):** A captura de pacotes em baixo nível exige a abertura de *raw sockets*. O contêiner do IDS precisa dessas *capabilities* do kernel Linux liberadas no compose.
- **Volumes Mapeados (Bind Mounts):**
    - `volume_logs`: A Vítima grava os logs HTTP/App aqui. O IDS monta esse volume em modo "somente leitura" (read-only) para analisá-los (tailing).
    - `volume_db`: O IDS grava o arquivo SQLite (`alerts.db`) aqui. O Dashboard monta esse volume para ler e exibir os dados na interface web.

O Motor do IDS em Python (Concorrência)

O arquivo `core.py` atuará como o inicializador (entrypoint) do contêiner do IDS. O grande desafio aqui é que as detecções precisam ocorrer em tempo real e em paralelo (não-bloqueantes).

- **Múltiplas Threads:** O `core.py` importará as funções de cada módulo de segurança (`detect_portscan.py`, `detect_bruteforce.py`, etc.) e invocará cada uma dentro de sua própria `threading.Thread`.

    1. *Módulos de Rede* ficarão em um loop de `sniff()` do Scapy.
    2. *Módulos de Log* ficarão em um loop lendo as novas linhas dos arquivos (como um `tail -f`).
- **Padrão Producer-Consumer (Fila de Tarefas):** O SQLite não lida bem com gravações simultâneas de múltiplas threads (*database is locked*). Para garantir a integridade dos dados:

1. **Produtores:** Os 5 módulos de detecção atuam como produtores. Quando identificam um ataque, instanciam um dicionário/JSON com o alerta e o colocam em uma fila thread-safe (`queue.Queue` no Python).
2. **Consumidor:** O módulo `db.py` roda em uma thread dedicada apenas para ler essa fila. Ele consome os itens um por um e executa as instruções de `INSERT` no SQLite.

O Fluxo de Vida de um Alerta (Exemplo Prático)

Para contextualizar o trabalho de cada membro no fluxo do sistema:

1. **O Ataque:** O atacante dispara uma ferramenta (ex: Dirb/Gobuster) contra a porta 80 do contêiner `vitima`.
2. **O Log:** A aplicação Flask (`vitima/app.py`) processa os diretórios inexistentes e gera diversos códigos HTTP 404, gravando isso em `/var/log/vitima/access.log`.
3. **A Detecção:** A thread rodando o `detect_fuzzing.py` (dentro do `ids/core.py`) está varrendo as novas linhas desse log. Ela identifica que o limiar (threshold) de erros 404 do mesmo IP excedeu o limite.
4. **A Fila:** O módulo cria o objeto de alerta e invoca `fila_alertas.put(novo_alerta)`.
5. **A Persistência:** A thread do banco de dados dá um `.get()` na fila e grava o evento no SQLite que está no volume compartilhado.
6. **A Visualização:** A aplicação Flask do `dashboard/app.py` faz um `SELECT` no banco e atualiza a interface web, renderizando o alerta para o usuário.

---

3. Sugestão de Divisão de Tarefas

Como a ideia é que todo mundo trabalhe em paralelo, pensei em dividir em "Módulos". Tentei alocar de um jeito que fizesse sentido, mas por favor, digam se preferem trocar!

Integrante	Módulo de Segurança (O que vai detectar)	Componente do Sistema (O que vai programar)	MITRE
Enzo Kuwamoto	**Ataque de Força Bruta:** Detectar múltiplas falhas de login (ex: Hydra) lendo os logs de acesso.	**Dashboard:** Interface web em HTML/Flask para ler o SQLite e exibir os alertas para o professor ver.	T1110
Henrique Hida	**Port Scan:** Detectar varredura de portas na rede (ex: Nmap) via pacotes.	**Infraestrutura:** Criar o `docker-compose` e configurar as redes isoladas do laboratório.	T1046
Andrey Marques	**SQL Injection / XSS:** Detectar caracteres anômalos (`' OR 1=1`) nas requisições do log.	**Banco de Dados:** Estruturar as tabelas do SQLite e a classe Python que grava os alertas de forma segura.	T1190
Matheus Schalch	**Fuzzing de Diretórios:** Detectar robôs caçando pastas escondidas (ex: Dirb/Gobuster) por excesso de erros 404.	**App Vítima:** Criar o app em Flask que será atacado, gerando as rotas e gravando os logs que o IDS vai ler.	T1595
Thomas Moises	**DoS (Negação de Serviço):** Detectar enxurrada de pacotes / SYN Flood travando a rede (ex: hping3).	**Núcleo do IDS:** O script principal que roda em background, inicializa as detecções e organiza o fluxo.	T1498

4. O que estudar na primeira semana

Se toparem essa divisão, o primeiro passo para cada um é:

1. Pesquisar sobre o seu **ataque específico** e como hackers usam as ferramentas citadas.
2. Entender como a sua detecção será mapeada no **MITRE ATT&CK**.
3. Estruturar a base do seu **componente do sistema**.

---

5. Cronograma Sugerido (Prazo: 06/11)

- **Semana 1 (até 16/10):** Laboratório Docker e Vítima de pé. Pesquisa individual do seu ataque.
- **Semana 2 (até 23/10):** Cada um tenta fazer o seu script detectar o seu ataque isoladamente rodando no terminal.
- **Semana 3 (até 30/10):** Integração! Os scripts mandam pro Core/Banco de Dados e o Dashboard exibe.
- **Semana 4 (até 06/11):** Tirar prints de evidências, montar o documento final e ensaiar a apresentação.